
Paraphrase Identification Using Cross-Attention for Large Online Community Platforms

Darbes Adrien - 11301005
Demers Olivier - 11322094
Housni Soheil - 11304451

Abstract

Duplicate questions are common on large online discussion platforms (e.g., Quora), fragmenting information and degrading search quality. We study paraphrase identification on the Quora Question Pairs (QQP) benchmark using Sentence-BERT (SBERT) as an efficient Siamese baseline. We then propose an interaction-aware variant (CA-SBERT) that inserts a Cross-Attention module before pooling, allowing token-level alignment across the two questions while preserving a shared-encoder structure. Due to computational constraints, we train and tune both models on a 15,000-pair subset and perform Bayesian hyperparameter optimization using validation F1 as the objective. Our results show that CA-SBERT matches but does not consistently outperform the SBERT baseline under limited data, suggesting that the added capacity increases optimization sensitivity and may require larger-scale training to yield clear gains.

1 Introduction

Online discussion platforms such as Reddit and Quora often contain many duplicate questions, where users ask the same query using different wording. This fragments answers across multiple threads and makes information retrieval harder.

We frame this as a paraphrase identification problem for duplicate-question detection. In this project, we compare an efficient SBERT baseline to a modified SBERT architecture that introduces cross-sentence interactions.

Contributions. (i) We implement a strong SBERT baseline for QQP paraphrase identification under constrained compute. (ii) We propose CA-SBERT, which inserts a Cross-Attention module before pooling to inject token-level interactions while preserving a shared-encoder structure. (iii) We compare both models under matched training budgets using Bayesian hyperparameter optimization and report performance and optimization sensitivity.

2 Related Work

Paraphrase detection refers to the task of detecting whether two sentences or texts express the same meaning. On large community platforms such as Quora, it specifically refers to duplicate question detection. Given two user-posted questions, we predict whether they are semantically equivalent. The Quora Question Pairs (QQP) (3) is the standard benchmark for this task, with over 400k labeled question pairs, where the label indicates whether the sentences are paraphrases of each other. Our research uses a reformatted subset of this dataset which is available on Hugging Face with 149k labeled pairs. (5)

Multiple studies have been conducted on QQP. For example, Ansari et al. (2020) (1) and several other Kaggle studies compared logistic regression, random forests, SVMs and gradient-boosted trees

for duplicate question detection on the QQP dataset. Deep learning approaches such as Autoencoders, CNNs and RNNs were also experimented with. A recent survey of deep learning techniques used in paraphrase identification by Zhou, Qiu, Liang, and Acuña (2022) offers a comprehensive review of paraphrase identification with deep learning. (9).

Other deep learning techniques such as Transformer-based language models are also efficient in sentence-pair modeling. BERT (Bidirectional Encoder Representation from Transformers) by Devlin, Chang, Lee and Toutanova (2019) (2) is a deep bidirectional transformer pre-trained on large unlabeled corpora using masked-language modeling and next-sentence prediction. For paraphrase identification, BERT is used as a cross-encoder: the two sentences are concatenated with special tokens and processed jointly.

Reimers and Gurevych (2019) (4) then proposed Sentence-BERT (SBERT) by adapting BERT into a Siamese network that outputs fixed-size sentence embeddings. In SBERT, two sentences are encoded independently by weight-sharing BERT encoders. SBERT makes semantic similarity, clustering, and duplicate detection much more efficient: each sentence is embedded once, and the same vector can be reused across all pairs. The trade-off is that both sentences are encoded independently, so the model never leverages token-level alignments between them during encoding; all cross-sentence interaction is pushed to the final similarity or classification layer.

To improve interaction modeling, we propose an SBERT architecture that implements a Cross-Attention layer. Cross-attention was popularized by the Transformer architecture of Vaswani et al. (2017) (7) and allows tokens of a particular question to attend to the tokens of another and vice versa, hence the above-named Cross-Attention.

Overall, the literature reveals a trade-off: cross-encoder Transformers provide strong accuracy through Cross-Attention, whereas Siamese encoders like SBERT offer the efficiency needed for real-world duplicate detection but lack explicit token-to-token alignment.

Our work addresses this gap by using the SBERT architecture and inserting a Cross-Attention layer between the BERT encoder and the pooling layer. This design preserves the shared-encoder efficiency of SBERT while enabling each token in one question to attend to relevant tokens in the other, resulting in interaction-aware sentence representations tailored to paraphrase identification on QQP.

3 Approach

To address the paraphrase detection problem, we leveraged the SBERT architecture (4). This architecture takes as input two sentences, and produces sentence embeddings for each of them that can then be compared to measure semantic similarity. SBERT is a *siamese* architecture, meaning that the transformer encoders processing Sentence A and Sentence B share the same parameters. This is necessary to ensure that the similarity computed between two sentences is symmetrical. First, each sentence is passed independently through a BERT encoder, corresponding to the encoder part of the classical Transformer architecture (7). This enables the model to produce an enriched contextual embedding for each token of each sentence. For each sentence, token embeddings are then fed to a pooling layer to be averaged and produce a sentence embedding. The sentence embeddings of both sentences are then concatenated along with their absolute difference. This concatenated vector then goes through a feed-forward neural network to proceed to the final classification.

In the following subsections, we will describe in detail the functioning of both our Baseline SBERT model and our improved SBERT model with Cross-Attention.

3.1 Baseline Model Architecture

Our baseline model first encodes the two input sentences independently using a shared pretrained SBERT encoder (weights sharing) (4) that output sentence embedding for each sentence. The resulting sentence embeddings are combined (concatenation and element-wise absolute difference) and passed to an MLP that outputs logits for the two classes: paraphrase and non-paraphrase.

3.1.1 SBERT Encoder

Each of the two SBERT encoder of the baseline model architecture takes as input one sentence. The two input sentences go through the same process described below independently.

Word, Positional, and Token Type Embeddings

A word embedding of dimension d_m is computed by the SBERT encoder for each token of the sentence(7). These embeddings are enriched by adding a positional embedding encoding each token's position (7) in the sequence and a token type embedding encoding which sentence a token belongs to when two sentences are concatenated with [SEP]. All three embeddings (word, position, token type) are already learnt by the pretrained SBERT encoder.

Multi-Head Self-Attention

The token embeddings are then processed by a Multi-Head Self-Attention module, which models how each token attends to every other token in the sequence. (7) Let Q, K, and V denote the Queries, Keys, and Values. The Query corresponds to the embedding representation of a focus word asking how it should attend other words; Keys are embedding representations of the relevance of candidate words to the Query; Values are embedding representations of the actual information that each word provides to the Query. Let $X \in \mathbb{R}^{|S| \times d_m}$ denote the matrix of token embeddings for a sentence of length $|S|$, and d_m be the token embedding dimension of the model. A Transformer encoder uses scaled dot-product attention (7):

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) \cdot V,$$

where Queries, Keys, and Values are obtained via learned linear projections $Q = XW_Q$, $K = XW_K$, and $V = XW_V$, with W_Q , W_K , W_V being respectively the weight matrices for Queries, Keys, and Values. d_k is the embedding dimension of Queries and Keys. The resulting output is a contextualized representation in which each token aggregates information from other tokens according to the attention weights.

Using multiple attention heads allows the model to learn different semantic relations. Multi-head attention repeats this computation in parallel for h heads (with distinct projection matrices W_{Q_i} , W_{K_i} , W_{V_i} for each head i), concatenates the head outputs, and applies a final linear projection.

$$h_i(Q_i, K_i, V_i) = \text{softmax}\left(\frac{Q_i K_i^\top}{\sqrt{d_k}}\right) \cdot V_i$$

This produces an attention matrix h_i for head i . Concatenating all heads yields a matrix $H = [h_1, \dots, h_n]$, which is projected through a final weight matrix W_O to restore the dimension $|S| \times d_m$ and to capture the last semantic relations of the concatenation.

This allows the model to capture multiple types of relations simultaneously (7). In practice, attention is followed by residual connections with input token embeddings, layer normalization, and a feed-forward network, producing enriched contextual token embeddings of dimension d_m for each token of the sequence.

Pooling

Following the encoding phase, we apply a mean pooling operation over the token embeddings of the sentence to obtain a sentence-level embedding of dimension d_m .(4)

3.1.2 Concatenation

The SBERT encoder is applied to each of the two input sentences: sentence A and sentence B. This results in two enriched embedding vectors for our sentence pairs: an embedding vector u for sentence A and an embedding vector v for sentence B. The two sentence embeddings are then concatenated alongside their absolute difference. (4)

$$c = \begin{pmatrix} u \\ v \\ |u - v| \end{pmatrix}, \quad c \in \mathbb{R}^{3 \times d_m}$$

3.1.3 Feed-Forward Neural Network

This concatenated vector c is then passed through an MLP that produces raw logits for each classes: *paraphrase* and *non-paraphrase*. These outputs are then used for classification.

3.2 CA-SBERT model

As mentioned above, the basic SBERT architecture leveraged by our baseline model encodes Sentence A and Sentence B independently (4). However, it is reasonable to think that the interactions between those two sentences may help to identify whether they are similar or not. Therefore, we introduce a Multi-Head Cross-Attention module (7) before the pooling operation to recognize token-level interactions across sentences. The rest of the architecture of this model is identical to that of the Baseline model.

Cross-attention (across a sentence pair).

In self-attention, Q , K , and V are all derived from the same sentence (i.e., from the same X), so each token attends to other tokens within its own sentence. Comparatively, in Cross-Attention, Queries come from one sentence while Keys and Values come from the other. (7) For example, letting X_A and X_B be the token-embedding matrices for sentences A and B:

$$Q = X_A W_Q, \quad K = X_B W_K, \quad V = X_B W_V,$$

so tokens in sentence A attend directly to tokens in sentence B. We also compute the reverse direction ($B \rightarrow A$). Therefore, our implementation of the Multi-Head Cross-Attention module before each pooling layer is identical to the one of the Multi-Head Self-Attention described above, except for the way Queries, Keys, and Values are assigned.

This additional module enriches token embeddings by modeling direct interactions between the two sentences. Consequently, the resulting pooled sentence embeddings carry richer cross-sentence semantic information.

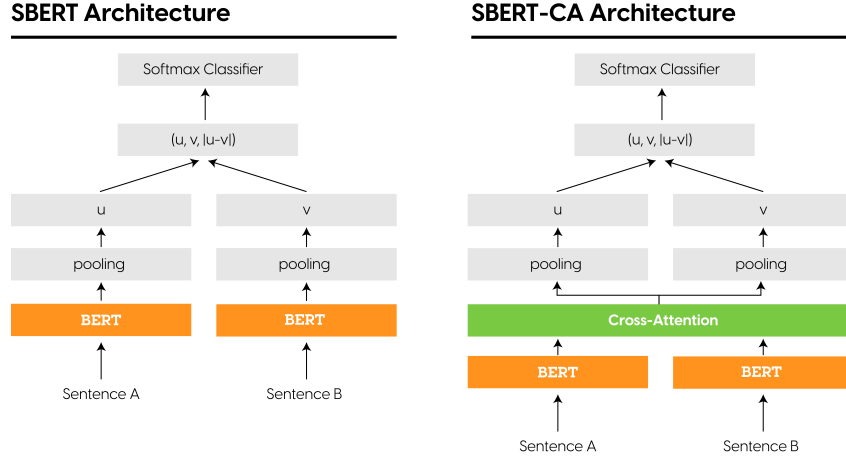


Figure 1: Baseline SBERT vs. SBERT with Cross Attention

4 Experimental Setup

This subsection describes the main training configurations that were used to train these models as well as the fixed hyperparameters and design choices that were made to compare the Baseline SBERT architecture to the Cross-Attention SBERT architecture.

4.1 Model Architectures

In our Baseline model, we first instantiate two pretrained SBERT encoders for each of the input sentence using *paraphrase-MiniLM-L6-v2* (6), which uses a 6-layer Transformer backbone and

produces 384-dimensional sentence embeddings via mean pooling. The two sentence embeddings are then concatenated in a same vector alongside their absolute difference as described above. This concatenated vector c is then passed through an MLP. The MLP is composed of blocks, each one being constituted of a linear layer, a normalization layer, a GELU activation function, and a dropout, except for the last layer (linear) of dimension $d_{out} = 2$ that produces raw logits for each classes: *paraphrase* and *non-paraphrase*. It is to be noted that the softmax function could have been used to transform those logits into probabilities, but this operation is directly applied in the Cross-Entropy function of PyTorch. That is why our network directly output logits and not probabilities. This is also the case in (4).

Our CA-SBERT model uses the same encoder and classifier pipeline as the Baseline model, but inserts a Cross-Attention module before the pooling of each of the two sentence.

Our optimizer for both models was AdamW. We chose AdamW instead of Adam as it is usually appropriate when overfitting is a concern and for NLP tasks such as ours. The learning rate used in conjunction with our optimizer was variable; specifically, we used a linear scheduling strategy with a warmup. In terms of criterion, we used Cross-Entropy for both models.

All iterations were performed for a maximum of 10 epochs, with early stopping based on a patience of 2 epochs. The data split was performed as follows: 74.8 % training, 10.2 % validation, 15 % test. Both models used a sample of size 15,000 before splitting as described above. While this sample is much smaller than our original data set, this choice is justified due to computational limitations.

4.2 Bayesian Optimization for Hyperparameter Tuning

Hyperparameter search is quite challenging in scenarios where the computational cost of training the model is high or when the search space is of high dimension, which was the case for our models. As computational resources were limited, this led us to use Bayesian Optimization (BO)¹.

BO is a sample-efficient family of optimization algorithms used for black-box functions. In both our models, we used the validation F1 score as the objective function, as this metric is useful for classification problems where there is significant imbalance in categories. In our dataset, 37 % of the phrases were paraphrases (label = 1) showing significant imbalance. The variables of this function are the hyperparameters themselves: $X_{hp} \in \mathbb{R}^n$, where n is the number of hyperparameters in the BO search². The BO model utilizes a surrogate model to approximate the objective function. We used a Gaussian Process (GP) as our surrogate model as proposed by (8). To select the next trial vector x_t , we use the Log Expected Improvement (LogEI) as the acquisition function, which selects a point that should be a good candidate to maximize the objective function given that we've gathered at time t the set $D_{t-1} := \{(x_i, y_i)\}_{i=1}^{t-1}$ of evaluated points. We can select the next vector x_t as such:

$$x_t = \arg \max_{x \in X_{hp}} \text{LogEI}(x | D_{t-1}, \text{GP}_{t-1})$$

Algorithm 1 presents the BO algorithm used to search over the set of hyperparameters. We used an initial sampling set of size $t_{init} = 10$ that was uniformly sampled across each respective range to fit the GP as follows:

$$D_{10} := \{(x_i, y_i)\}_{i=1}^{10}$$

The subsequent steps $t = 11, 12, \dots, T$ were sampled as shown in the BO search algorithm (see 1). The maximum number of iterations for the BO search (T) was set to 40. We also implemented early stopping. Let $S_t \in \{0, 1\}$ be the early stopping indicator at step t :

$$S_t = \begin{cases} 1 & \text{if } \max(\{y_i\}_{i=t-5}^t) - \min(\{y_i\}_{i=t-5}^t) < 0.01 \\ 0 & \text{otherwise} \end{cases}$$

Table 1 presents the lower bounds (LB) and upper bounds (UB) defining the search space for each hyperparameter in the baseline and Cross Attention (CA) models. "Nb frozen SBERT layers" represents the number of layers that have frozen parameters in the pre-trained SBERT. The "Nb of hidden layers SBERT" parameter indicates the number of hidden layers to append after the SBERT

¹We used the BoTorch python library for the BO.

²The hyperparameters used in the search included discrete hyperparameters we considered as continuous in the BO process. They were floored before actual usage in the training loop.

Algorithm 1 Bayesian Optimization for Hyperparameter Tuning

```
1: Input  $X_{ub}$ , the set of search upper bounds of the hyperparameters
2:    $X_{lb}$ , the set of search lower bounds of the hyperparameters
3:    $X_{discrete} \subset X_{hp}$ , the set of discrete hyperparameters
4:
5: Initialize an experience buffer  $D_t$ 
6: for  $t = 1$  to  $t_{init}$  do
7:   Sample  $x_t \sim U(X_{lb}, X_{ub})$ 
8:    $x_{t,i} \leftarrow \lfloor x_{t,i} \rfloor \quad \forall i \in X_{discrete}$ 
9:    $y_t \leftarrow f(x_t)$ 
10:  Store  $(x_t, y_t)$  in experience buffer  $D_t$ 
11: Fit  $GP_{t_{init}}$  on  $D_{t_{init}}$ 
12:
13: for  $t = t_{init}+1$  to  $T$  do
14:    $x_t \leftarrow \arg \max_{x \in X_{hp}} \text{LogEI}(x_t | D_{t-1}, GP_{t-1})$ 
15:    $x_{t,i} \leftarrow \lfloor x_{t,i} \rfloor \quad \forall i \in X_{discrete}$ 
16:    $y_t \leftarrow f(x_t)$ 
17:  Store  $(x_t, y_t)$  in experience buffer  $D_t$ 
18:  Fit  $GP_t$  on  $D_t$ 
19:
20:  if  $\max(\{y_i\}_{i=t-5}^t) - \min(\{y_i\}_{i=t-5}^t) < 0.01$  then
21:    break
```

embeddings. Including hyperparameters for the neuron counts of all three potential layers, alongside the parameter for the number of layers itself, allows us to vary the architecture while keeping the search space dimension fixed. This is necessary because BO requires X_{hp} to have a constant dimension across iterations. Consequently, if the layer count parameter suggests a single layer (e.g., 1.0), the values sampled for the second and third hidden layers are simply ignored during model construction. The last three hyperparameters in Table 1 apply only to the CA model and represent the architectural parameters for the MLP within the Cross Attention layer. As previously mentioned, this setup provides control over the network depth while maintaining a fixed dimensionality for X_{hp} .

Hyperparameters	Baseline model		Cross Attention model	
	LB	UB	LB	UB
Learning Rate	$1 \cdot 10^{-4}$	$3 \cdot 10^{-2}$	$1 \cdot 10^{-5}$	$1 \cdot 10^{-3}$
Weight Decay	0.001	0.1	0.001	0.1
Dropout Probability	0.0	0.5	0.0	0.5
Batch Size	16.0	96.9	16.0	96.99
Nb frozen SBERT layers	3.0	6.0	3.0	6.0
Nb of hidden layers in SBERT	1.0	3.9	1.0	3.9
Neurons for hidden 1 in SBERT	64.0	256.9	64.0	256.9
Neurons for hidden 2 in SBERT	64.0	128.9	64.0	128.9
Neurons for hidden 3 in SBERT	16.0	64.9	16.0	64.9
Nb of hidden layers in CA	-	-	1.0	2.9
Neurons for hidden 1 in CA	-	-	600.0	900.9
Neurons for hidden 2 in CA	-	-	400.0	600.9

Table 1: Hyperparameter lower and upper search space bounds for the baseline and cross attention model

4.3 Hyperparameters

Table 2 shows the hyperparameters that were selected during the BO search. The baseline model was less sensitive to hyperparameters compared to the CA model. Most sets of hyperparameters led to around the same validation F1 scores. The CA Model had a lot of variance in terms of validation

F1 score, showing very high sensitivity to hyperparameters. That is, only a small subset of the hyperparameter search space led to decent results. The selection of the hyperparameter set was based on $x^* = \arg \max_{(x,y) \in D} (y)$.

Hyperparameters	Baseline Model	Cross Attention Model
Learning Rate	$1.2 \cdot 10^{-2}$	$5.2 \cdot 10^{-5}$
Weight Decay	0.0086	0.065
Dropout Probability	0.41	0.23
Batch Size	58	76
Nb frozen SBERT layers	6	3
Nb of hidden layers in SBERT	3	1
Neurons for hidden 1 in SBERT	116	198
Neurons for hidden 2 in SBERT	86	-
Neurons for hidden 3 in SBERT	30	-
Nb of hidden layers in CA	-	2
Neurons for hidden 1 in CA	-	801
Neurons for hidden 2 in CA	-	489

Table 2: Hyperparameter set selected during BO loop based on the highest validation F1 score for the baseline model and the cross attention model

5 Results

Following our experiments, several notable results were observed. First, we observed that the regularization hyperparameters, especially the dropout probability and the weight decay, are high for both models. This can be explained by the fact that these models are complex due to the SBERT encoder, which contains 22,713,216 parameters. As our model is trained on only 11,200 data points, it can overfit quickly, which is why strong regularization was required.

For the same reason, training stopped early for both models due to early stopping (7 epochs for the baseline model and 5 for the Cross-Attention model). This demonstrates that performance improvements started to plateau quickly, slightly earlier for the Cross-Attention model.

We also observed that the number of frozen encoder layers from the pretrained SBERT model is twice as high in the baseline model as in the Cross-Attention model. In the baseline model, every layer of the pretrained encoder remained frozen, which helps prevent overfitting given our limited dataset. However, in the Cross-Attention model, only three layers were kept frozen. This is because adding a Cross-Attention module, which captures inter-sentence token relations, requires more adaptation of the encoder weights to the data.

Moreover, the Cross-Attention model is more sensitive to training than the baseline one, as reflected in the learning-rate ranges. To obtain good performance, the baseline model required a learning rate in the order of 10^{-2} , whereas the Cross-Attention model required 10^{-5} . This is due to the fact that the Cross-Attention model is significantly more complex than the baseline one, and therefore more difficult to train.

In short, the Cross-Attention model is harder to train overall: it is more complex, tends to overfit quicker, and requires more adaptation to the data.

Test metrics	Baseline model	Cross-Attention model
Loss	0.312	0.375
Accuracy	0.863	0.845
F1	0.823	0.817

Table 3: Test metrics: Baseline SBERT vs. CA-SBERT

Finally we observed that the validation loss, accuracy, and F1 score, for both the baseline model and the Cross-Attention model followed the same trends. The final epoch validation F1 score for the baseline model is 0.832856, while it is 0.834559 for the Cross-Attention model. These two scores are almost identical, with a very slight advantage for the Cross-Attention model.

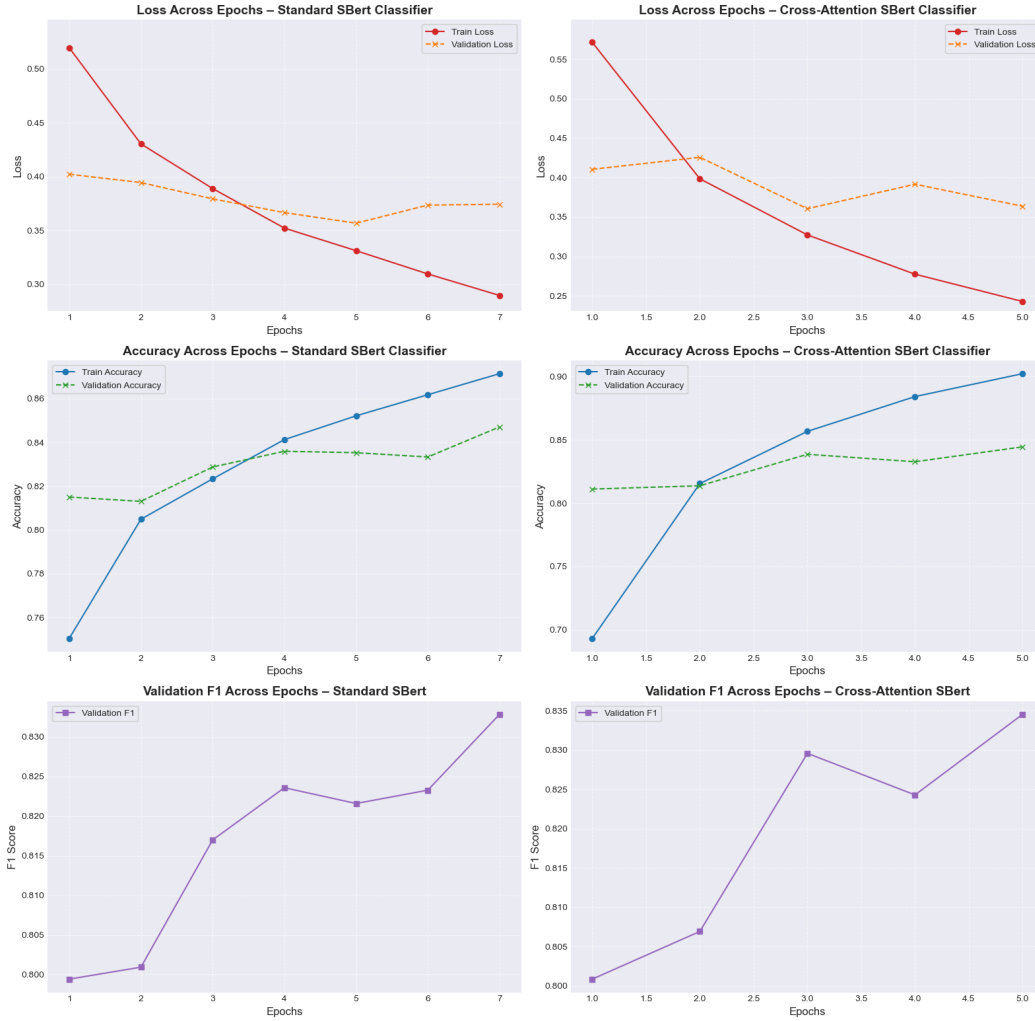


Figure 2: Metrics comparison: Baseline SBERT vs. CA-SBERT

However, on the test set, the baseline model performed slightly better, even though the metrics are very close. This can be explained by the fact that the Cross-Attention model is much more complex than the standard one and would require more data to be trained efficiently. The training was done on only 11,200 data points, which is not sufficient for such a large model.

6 Discussion and Conclusion

Our experiments showed that the standard SBERT model and the Cross-Attention model share very similar performances, with slightly better results for the standard architecture. To exploit the full potential of Cross-Attention, it would need to be trained on a larger dataset because of its high complexity. We were limited in our computation capability. Indeed, while our data set contained 149k data points, we were only able to use roughly 10% of the whole data set for our training (15k observations). Consequently, our models based on a highly complex SBERT pretrained encoder with 22 713 216 parameters tended to overfit quickly due to the small size of the training set. To face this issue, various and heavy regularization methods were employed. This overfitting limitation on a small dataset might lead us toward the integration of a cross-encoder which would unify the siamese architecture. However, such an architecture might prevent us from outputting individual sentence embeddings that can be useful for other tasks.

References

- [1] Ansari, N. and Sharma, R. (2020). Identifying Semantically Duplicate Questions Using Data Science Approach: A Quora Case Study. *arXiv preprint* arXiv:2004.11694. Available at <https://arxiv.org/abs/2004.11694>.
- [2] Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv preprint* arXiv:1810.04805. Available at <https://arxiv.org/abs/1810.04805>.
- [3] Quora. Quora Question Pairs. Kaggle competition, 2017. Available at <https://www.kaggle.com/competitions/quora-question-pairs>.
- [4] Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Available at <https://arxiv.org/abs/1908.10084>.
- [5] Sentence Transformers. Quora Duplicate Questions (sentence-transformers/quora-duplicates). Hugging Face Datasets. Available at <https://huggingface.co/datasets/sentence-transformers/quora-duplicates>.
- [6] Sentence Transformers. (n.d.). *paraphrase-MiniLM-L6-v2* [Machine learning model]. Hugging Face. Retrieved December 12, 2025, from <https://huggingface.co/sentence-transformers/paraphrase-MiniLM-L6-v2>.
- [7] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention Is All You Need. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*. Available at <https://arxiv.org/abs/1706.03762>.
- [8] Wu, J., Chen, X.-Y., Zhang, H., Xiong, L.-D., Lei, H., and Deng, S.-H. (2019). Hyperparameter Optimization for Machine Learning Models Based on Bayesian Optimization. *Journal of Electronic Science and Technology*, 17(1), 26–40. Available at <https://www.sciencedirect.com/science/article/pii/S1674862X19300047>.
- [9] Zhou, C., Qiu, C., Liang, L., and Acuna, D. E. (2022). Paraphrase Identification with Deep Learning: A Review of Datasets and Methods. *arXiv preprint* arXiv:2212.06933. Available at <https://arxiv.org/abs/2212.06933>.